MULTITHREAD MAYHEM // OS & CONCURRENCY

WEEK 03

Pthreads API & Classic Problems

Problem Set

Parallel stats, semaphores, readers-writers, thread pool.

Weeks 1 and 2 gave you the tools.
This week you use them properly.
Return results from threads, switch to semaphores,
and solve the readers-writers pattern.

Then build a minimal thread pool.

INSIDE

PROBLEM 00

The week, in four problems.

Four problems for the week. You will return values from threads, switch from condition variables to semaphores, build a readers-writers lock, and finish with a minimal thread pool. Problems 1 and 2 are the minimum; 3 and 4 are there if you have time.

Overview

Start with the theory. Then run the example programs in `code/` at least once. Pay particular attention to the thread return-value example, the semaphore example, and the readers-writers example; those patterns show up directly in the problem set.

#	Problem	Focus	Required?
01	Parallel statistics	Thread return values + aggregation	Yes
02	Semaphore bounded buffer	Semaphore-based producer-consumer	Yes
03	Readers-writers from scratch	Shared-read / exclusive-write coordination	If attempted
04	Thread pool	Worker threads + task queue	Optional

> ***Before you start:*** *read* `theory.pdf` *and run the examples in* `code/` *at least once. Problem 1 leans on thread return values. Problem 2 is the semaphore version of producer-consumer. Problems 3 and 4 are the deeper synchronization exercises.*

How to compile and run

Inside `code/` there is a `Makefile` that builds the week's example programs:

```
cd week-3/code
make               # builds all four examples
./thread_lifecycle
./return_values
./semaphores
./readers_writers
make run-all       # builds and runs all four in order
make clean         # removes compiled binaries
```

Compile flag required: `-pthread -std=c11`. If something hangs, look for a missing signal, an unreleased lock, or a thread that outlived `main`.

Suggested plan for the week

Day	What to do
1	Read sections 1 and 2 of theory.pdf. Compile and run code examples 01 and 02.

2	Read section 3 of theory.pdf. Run code example 03.
3	Read section 4 of theory.pdf. Run code example 04.
4	Start the problem set.
5	Finish the problem set and write your report.

Read, then run, then solve — same as the previous weeks.

PROBLEM 01

Parallel statistics

Split a large input across multiple threads, collect each worker's result safely, and combine the partial answers in the main thread. This problem is about returning data from threads without dangling pointers.

What to do

Write a program that computes useful statistics over a large data set using several threads. Each worker should process a slice of the input and return its partial result. The main thread joins every worker, combines the partial results, and prints the final statistics.

Choose a sensible set of statistics — for example sum, minimum, maximum, and average. The exact set is up to you, but it should be clear what each thread computed and how the final answer was assembled.

Requirements

Use `pthread_join` to collect return values or write into a shared output structure that the main thread reads after joining. Do not return pointers to stack variables. Test with different thread counts and verify that the results stay correct every run.

```
// Sketch only
typedef struct {
    int start, end;
    long sum;
    int min, max;
} worker_result_t;

void *worker(void *arg) {
    worker_result_t *r = arg;
    /* compute partial stats */
    return NULL;
}

// main() joins all workers, then combines results
```

***Report note:** mention which statistics you computed, how you collected thread return values, and whether the answer was stable across repeated runs.*

What to submit

`parallel_stats.c` and a short note in your report.

PROBLEM 02

Semaphore bounded buffer

Rebuild the Week 2 bounded buffer using semaphores instead of a condition-variable solution. The buffer should block correctly when it is full or empty, and it should work with multiple producers and multiple consumers.

What to do

Implement a thread-safe circular buffer that supports put and get operations. Use semaphores to count available items and free slots, and a mutex to protect the actual buffer state. Your buffer should block on put when full and block on get when empty.

The goal is to understand how a semaphore-based design differs from the Week 2 condition-variable version. In your report, explain that difference in your own words.

```
void buffer_init(int capacity);
void buffer_put(int item);   /* blocks if full */
int  buffer_get(void);       /* blocks if empty */
void buffer_destroy(void);

// typical layout
sem_t empty;
sem_t full;
pthread_mutex_t mutex;
int *data;
int in, out, count;
```

Test it

Create multiple producers and multiple consumers. Make the buffer tiny as well as large. Verify that the total number of items consumed equals the total number produced, and that the sums match too.

> *Semaphores encode the count directly, so you should not need a* `while`*-loop for spurious wakeups the way you did with condition variables.*

What to submit

`sem_buffer.c` and a short paragraph in your report describing how the semaphore version differs from the Week 2 version.

PROBLEM 03

Readers-writers from scratch

Build a synchronization primitive that allows many readers at once but only one writer at a time. This is the classic shared-read / exclusive-write pattern.

The problem

Many real systems allow concurrent reads but require exclusive writes. A database, a cache, or a configuration store can often serve many readers simultaneously, but any writer must stop the readers while it updates the shared state.

Implement your own readers-writers solution from lower-level synchronization primitives. The important property is simple: multiple readers may enter together, but writers must be exclusive.

Minimum interface

```
void rw_init(void);
void reader_lock(void);
void reader_unlock(void);
void writer_lock(void);
void writer_unlock(void);
void rw_destroy(void);
```

A coarse-grained solution is acceptable. If you choose a preference policy, document it. A small stress test should verify that readers overlap, writers do not overlap with readers, and the program does not deadlock.

> *Do not make the lock live inside each node or each reader. The coordination state belongs with the shared resource, not with individual operations.*

What to submit

`rw_lock.c` and, if attempted, a brief note describing the policy you implemented and how you tested it.

PROBLEM 04

Thread pool

Finish the week by building a minimal worker pool: a fixed set of threads, a shared task queue, and a way to wait until all queued work has been completed.

The problem

Create a pool with a fixed number of worker threads. The workers wait for tasks, run them, and keep going until the pool is shut down. This pattern is everywhere: servers, pipelines, background jobs, and game engines.

You do not need a production-quality implementation. A small, correct pool is enough as long as it can accept tasks, execute them once, and shut down cleanly.

Suggested API

```
void pool_init(int workers);
void pool_submit(void (*fn)(void *), void *arg);
void pool_wait(void);
void pool_destroy(void);
```

Test with many tiny tasks. Make sure every task runs exactly once. If you add a result or counter, verify that the final total matches the number of tasks submitted.

What to submit

`thread_pool.c` and a short note in your report about how your queue and worker coordination work.

> *A minimal pool is fine. The important part is that the workers sleep when there is no work and wake up reliably when a task arrives.*

PROBLEM 05

Submission

Keep the source files ready for the Google Form your mentor will share. The minimum submission is Problems 1 and 2; Problems 3 and 4 are optional unless your mentor asks for them.

Files to keep ready

Problem	File to keep ready
Problem 1 — Parallel statistics	parallel_stats.c
Problem 2 — Semaphore bounded buffer	sem_buffer.c
Problem 3 — Readers-writers from scratch	rw_lock.c
Problem 4 — Thread pool	thread_pool.c

What your short report should cover

Which problems you attempted; for Problem 1, which statistics you computed and how you collected thread return values; for Problem 2, how the semaphore version differs from the Week 2 condition-variable version; and, if attempted, any notes for Problems 3 and 4.

Also note one thing that surprised you and anything you got stuck on. Honest observations matter more than polished prose.

What's expected

Expected	Notes
Clean compile	Use gcc -Wall -pthread -std=c11.
Termination	Every program should terminate normally.

Honest observations	Say if your parallel version was slower and think about why.

What's not expected

A production-quality thread pool. Familiarity with named semaphores. All four problems. Problems 1 and 2 are the minimum.

> *Do not submit compiled binaries — only `.c` source files. If a problem has multiple files, zip them together before uploading.*

Stuck?

Re-read `theory.pdf`. Compare your output to the sample outputs in `sample-outputs/`. If a thread is not terminating, check for a missing signal, an unreleased lock, or a detached thread that outlived `main`.

If you still get stuck, message your mentor.